

BEGINNER DEPLOYMENT GUIDE

JotFlow with Docker and GitHub Actions

Windows 10/11 • Docker Desktop • PostgreSQL • automatic deployment

Copy-paste guide for a new administrator

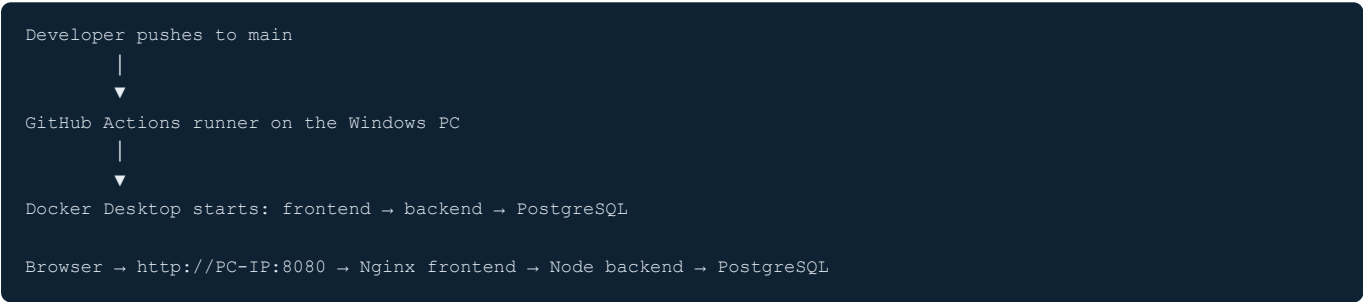
Version 1.0 • July 2026

1. What you are setting up

This guide deploys JotFlow on one Windows 10/11 PC. After setup, each push to GitHub's `main` branch automatically rebuilds and starts the application on that PC.

Term	Meaning
Container	A small isolated program. We run frontend, backend, and PostgreSQL containers.
Docker Compose	One configuration file that connects those containers.
GitHub Actions runner	A GitHub program installed on this PC. It receives deployment jobs.
Volume	Docker-managed disk space that survives container replacement.

Linux containers do not replace Windows. Docker Desktop runs them using its WSL 2 backend. Windows stays your PC's operating system.



The frontend forwards `/api` , `/uploads` , and `/socket.io` internally. Users only open one browser address.

2. Prepare the Windows PC

Step 1 — Check Windows

Open **PowerShell as Administrator**:

```
winver  
wsl --status
```

Use Windows 10 or 11 with virtualization enabled in BIOS/UEFI.

Step 2 — Install Docker Desktop

1. Download Docker Desktop for Windows from Docker's official website.
2. During installation choose **Use WSL 2 instead of Hyper-V**.
3. Restart if requested, open Docker Desktop, and wait for **Engine running**.

```
docker version  
docker compose version  
docker run --rm hello-world
```

Docker Desktop must remain running. The PC must be powered on, connected to the network, and not asleep for automatic deployments.

Step 3 — Find the PC IP

```
ipconfig
```

Copy the IPv4 Address from the active Wi-Fi or Ethernet adapter, for example `192.168.1.50`. Give this PC a DHCP reservation/static IP in the router so it does not change.

Step 4 — Open the firewall port

```
New-NetFirewallRule -DisplayName "JotFlow HTTP 8080" -Direction Inbound -Action Allow -Protocol TCP -LocalPort 8080
```

LAN users will open `http://<PC-IP>:8080`. Do not expose the port publicly until a domain and HTTPS reverse proxy are added.

3. Create the private production configuration

Secrets are kept only on the Windows PC, not in Git or GitHub Actions.

```
New-Item -ItemType Directory -Force C:\JotFlow
New-Item -ItemType Directory -Force C:\JotFlow\backups
notepad C:\JotFlow\production.env
```

Paste this and replace every `REPLACE_ME` value. The database password must match in both marked locations.

```
APP_PORT=8080
POSTGRES_USER=jotflow
POSTGRES_PASSWORD=REPLACE_ME_LONG_UNIQUE_PASSWORD
POSTGRES_DB=jotflow
DATABASE_URL=postgresql://jotflow:REPLACE_ME_SAME_PASSWORD@db:5432/jotflow
SESSION_SECRET=REPLACE_ME_RANDOM_SECRET
NODE_ENV=production
ALLOWED_ORIGIN=http://<PC-IP>:8080
PUBLIC_BASE_URL=http://<PC-IP>:8080
JOTFORM_API_KEY=REPLACE_ME
JOTFORM_API_KEY_GDMO=REPLACE_ME
JOTFORM_TEAM_ID=
JOTFORM_WEBHOOK_SECRET=REPLACE_ME
ENABLE_POLLER=1
POLL_INTERVAL_MINUTES=2
POLL_QUICK_SECONDS=1
POLLER_KEY_TYPE=gdm
```

Generate a session secret:

```
[Convert]::ToBase64String((1..48 | ForEach-Object { Get-Random -Maximum 256 })))
```

Never commit `C:\JotFlow\production.env` or paste it into an Actions log, issue, chat, or screenshot.

4. Install the GitHub Actions runner

1. Open your GitHub repository → **Settings** → **Actions** → **Runners** → **New self-hosted runner**.
2. Select **Windows** and **x64**. Keep GitHub's page open; it contains a one-time token.
3. Open PowerShell as the account that uses Docker Desktop and follow GitHub's download commands. The important registration command looks like this:

```
New-Item -ItemType Directory -Force C:\actions-runner
Set-Location C:\actions-runner
# Download and extract the ZIP specified by GitHub.
.\config.cmd --url https://github.com/<OWNER>/<REPOSITORY> --token <ONE_TIME_TOKEN>
.\run.cmd
```

After the first successful deployment, use GitHub's instructions to install the runner as a Windows service. Add its Windows account to the `docker-users` group, then sign out and back in.

GitHub should show the runner as **Idle** with labels `self-hosted`, `Windows`, and `x64`.

5. First deployment

The repository includes:

File	Job
<code>Dockerfile.frontend</code>	Build React and serve it with Nginx.
<code>Dockerfile.backend</code>	Run Node API and Socket.IO.
<code>docker-compose.production.yml</code>	Connect the three containers and persistent volumes.
<code>.github/workflows/deploy.yml</code>	Deploy automatically on every push to <code>main</code> .

Commit and push those repository files:

```
git add .
git commit -m "Add Docker production deployment"
git push origin main
```

Open the GitHub **Actions** tab. The workflow validates the local env file, builds images, starts PostgreSQL, runs the schema migration, starts frontend/backend, and calls the health endpoint.

When the job is green, open:

```
http://<PC-IP>:8080
http://<PC-IP>:8080/api/health/ready
```

The health response should include `"ok":true` and `"db":"up"`.

6. Everyday use and troubleshooting

Future releases need only:

```
git add .
git commit -m "Describe the change"
git push origin main
```

```
# View container state
docker compose --env-file C:/JotFlow/production.env -f docker-compose.production.yml ps

# Read backend logs (Ctrl+C stops viewing only)
docker compose --env-file C:/JotFlow/production.env -f docker-compose.production.yml logs -f backend

# Test locally on the PC
Invoke-WebRequest http://127.0.0.1:8080/api/health/ready -UseBasicParsing
```

Problem	Fix
Action stays queued	Docker Desktop and the self-hosted runner must be running; the PC cannot be sleeping.
Browser cannot connect	Check <code>ipconfig</code> , firewall port 8080, and container state.
Database health fails	Read database logs and ensure the two password occurrences in <code>production.env</code> match.
API/login blocked	<code>ALLOWED_ORIGIN</code> and <code>PUBLIC_BASE_URL</code> must equal the exact browser address, including IP and port.
Data seems missing	Do not run <code>docker compose down -v</code> ; <code>-v</code> deletes stored volumes.

7. Daily PostgreSQL backups

Create `C:\JotFlow\backup-postgres.ps1`:

```
$ErrorActionPreference = 'Stop'
$env:JOTFLOW_ENV_FILE = 'C:/JotFlow/production.env'
$repo = 'C:\actions-runner\_work\<REPOSITORY>\<REPOSITORY>'
$stamp = Get-Date -Format 'yyyy-MM-dd_HH:mm:ss'
$file = "C:\JotFlow\backups\jotflow_$stamp.sql"
Set-Location $repo
docker compose --env-file $env:JOTFLOW_ENV_FILE -f docker-compose.production.yml exec -T db pg_dump -U jotflow -d jotflow | Out-File -Encoding utf8 $file
Get-ChildItem C:\JotFlow\backups\*.sql | Where-Object LastWriteTime -lt (Get-Date).AddDays(-14) | Remove-Item
```

Replace `<REPOSITORY>`, then create a daily Task Scheduler task that runs:

```
powershell.exe -ExecutionPolicy Bypass -File C:\JotFlow\backup-postgres.ps1
```

Test it once and copy backups to a different machine/cloud location. A backup on the same PC does not protect against disk failure.

Final checklist

- ☐ Docker Desktop is running in Linux container mode.
- ☐ The PC has a stable IP and the firewall allows TCP 8080.
- ☐ `C:\JotFlow\production.env` exists only on the deployment PC.
- ☐ GitHub shows the self-hosted runner as Idle.
- ☐ A push to `main` produces a green deployment.
- ☐ `/api/health/ready` reports a healthy database.
- ☐ Daily backups are written and copied off the PC.